Protocol: PostgreSQL and MySQL for Bioprocess Lab Data

Install, set up, load, query, secure and back up a laboratory database

24 October 2026

1. Overview
2. Install
3. Create the database and users
4. Design and create the tables
5. Load data
   - 5.1 SQL inserts
   - 5.2 CSV files from instruments or Excel
6. Query the data
7. Control access
8. Back up and restore
9. Connect from Python
10. Troubleshooting

Appendix A – Full PostgreSQL script

Appendix B – Full MySQL script

1. Overview

This protocol sets up a relational database for bioprocess lab data (strains, bioreactor runs and time-course samples) in **PostgreSQL** and in **MySQL**. Every step shows both systems side by side.

How it was tested: all PostgreSQL commands, SQL and Python code were run on PostgreSQL 16.15 and gave the outputs shown. MySQL commands follow MySQL 8.0 syntax and were syntax-checked with a SQL parser, but were not run on a live MySQL server. Test them on your own server before relying on them.

Which one to choose:

Need	PostgreSQL	MySQL
Complex analytics, window functions, strict data checks	Strong	Good (8.0+)
Scientific data types (arrays, JSONB, ranges)	Strong	JSON only
Simple web apps, shared hosting	Good	Strong
Licence	PostgreSQL licence (permissive)	GPL (Community) or commercial
Default port	5432	3306

Workflow:

1. Install the server and client.
2. Create the database and user accounts.
3. Design and create the tables.
4. Load data (SQL inserts, CSV files, Python).
5. Query and summarize the data.
6. Control access.
7. Back up and restore.
8. Connect from Python for analysis.

2. Install

Platform	PostgreSQL	MySQL
Ubuntu / Debian	`sudo apt install postgresql postgresql-contrib`	`sudo apt install mysql-server`
macOS (Homebrew)	`brew install postgresql@16` then `brew services start postgresql@16`	`brew install mysql` then `brew services start mysql`
Windows	Installer from postgresql.org (includes pgAdmin)	MySQL Installer from mysql.com (includes Workbench)
Docker	`docker run -d --name pg -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres:16`	`docker run -d --name my -e MYSQL_ROOT_PASSWORD=secret -p 3306:3306 mysql:8`

Check the install:

```
psql --version                # PostgreSQL client
mysql --version               # MySQL client
```

On MySQL, run `sudo mysql_secure_installation` once to set the root password and remove test accounts.

3. Create the database and users

Use two accounts: an **admin** that owns the tables, and a **reader** that can only run queries. Replace the example passwords.

PostgreSQL (connect as the superuser: `sudo -u postgres psql`):

```
CREATE ROLE lab_admin WITH LOGIN PASSWORD 'change_me' CREATEDB;
CREATE DATABASE bioprocess OWNER lab_admin;
CREATE ROLE lab_reader WITH LOGIN PASSWORD 'change_me_too';
```

MySQL (connect as root: `sudo mysql` or `mysql -u root -p`):

```
CREATE DATABASE bioprocess CHARACTER SET utf8mb4;
CREATE USER 'lab_admin'@'localhost'  IDENTIFIED BY 'change_me';
CREATE USER 'lab_reader'@'localhost' IDENTIFIED BY 'change_me_too';
GRANT ALL PRIVILEGES ON bioprocess.* TO 'lab_admin'@'localhost';
FLUSH PRIVILEGES;
```

Connect as the admin:

```
psql -U lab_admin -d bioprocess -h localhost      # PostgreSQL
mysql -u lab_admin -p bioprocess                  # MySQL
```

4. Design and create the tables

Three linked tables hold the data. Each sample belongs to one run; each run uses one strain.

Table	Key columns	Purpose
`strain`	`strain_id`, `name`, `organism`, `genotype`	Organisms and engineered strains
`run`	`run_id`, `strain_id` → strain, `reactor`, `mode`, `temperature_c`, `ph_setpoint`, `started_at`	One bioreactor experiment
`sample`	`sample_id`, `run_id` → run, `time_h`, `od600`, `biomass_g_l`, `glucose_g_l`, `product_g_l`	Time-course measurements

Design rules used:

- **Primary keys** that number themselves: `SERIAL` in PostgreSQL, `INT AUTO_INCREMENT` in MySQL.
- **Foreign keys** so a sample cannot point to a run that doesn't exist. `ON DELETE CASCADE` removes a run's samples with it.
- **CHECK constraints** reject impossible values, such as negative concentrations or pH above 14. MySQL enforces these from version 8.0.16.
- **UNIQUE (run_id, time_h)** stops the same time point being entered twice.
- An **index** on `(run_id, time_h)` keeps time-course queries fast as data grows.

The main syntax differences:

Item	PostgreSQL	MySQL
Auto-number key	`SERIAL PRIMARY KEY`	`INT AUTO_INCREMENT PRIMARY KEY`
Exact decimal	`NUMERIC(6,3)`	`DECIMAL(6,3)`
Date and time	`TIMESTAMP`	`DATETIME`
Foreign key	inline `REFERENCES strain(strain_id)`	`FOREIGN KEY (...) REFERENCES ...` line, `ENGINE=InnoDB`

Create everything by running the full script for your system (Appendix A or B):

```
psql -U lab_admin -d bioprocess -f bioprocess_postgres.sql      # PostgreSQL
mysql -u lab_admin -p bioprocess < bioprocess_mysql.sql         # MySQL
```

5. Load data

5.1 SQL inserts

Small amounts of data can be typed as `INSERT` statements, as in the scripts. The same statements work in both systems.

```
INSERT INTO sample (run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l)
```

```
VALUES (1, 10, 6.4, 2.56, 0.0, 1.7);
```

5.2 CSV files from instruments or Excel

Save the sheet as CSV with a header row matching the column names:

```
run_id,time_h,od600,biomass_g_l,glucose_g_l,product_g_l
1,10,6.4,2.56,0.0,1.7
2,20,11.0,4.40,3.5,12.9
```

PostgreSQL (`\copy` runs from your computer, no server file access needed):

```
\copy sample (run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l)
  FROM 'new_samples.csv' WITH (FORMAT csv, HEADER true)
-- Result: COPY 2
```

MySQL (start the client with `mysql --local-infile=1 -u lab_admin -p bioprocess`; the server must also allow `local_infile`):

```
LOAD DATA LOCAL INFILE 'new_samples.csv'
INTO TABLE sample
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l);
```

A row that breaks a constraint (for example a negative glucose value, or a duplicate time point) is rejected with an error, so the bad data never gets in.

6. Query the data

These queries are in both scripts and give the same results in both systems.

Join the tables to see each sample with its strain and run:

```
SELECT r.run_id, s.name AS strain, r.mode, sa.time_h, sa.biomass_g_l, sa.product_g_l
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
ORDER BY r.run_id, sa.time_h;
```

Summarize each run (final titer, glucose used, product yield):

```
SELECT r.run_id, s.name AS strain,
       MAX(sa.product_g_l) AS final_titer_g_l,
       MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l) AS glucose_used_g_l,
       ROUND(MAX(sa.product_g_l) /
             NULLIF(MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l), 0), 3) AS yield_g_per_g
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
GROUP BY r.run_id, s.name;
```

run_id	strain	final_titer_g_l	glucose_used_g_l	yield_g_per_g
1	EC-WT	1.600	17.900	0.089
2	EC-SUC1	10.500	16.100	0.652

This yield uses only the glucose drop in the broth. For fed-batch runs it overstates yield, because fed glucose isn't counted; add a `feed` table to track it.

Specific growth rate between samples (window function, MySQL 8.0+):

```
SELECT run_id, time_h, biomass_g_l,
```

```
       ROUND(LN(biomass_g_l / LAG(biomass_g_l) OVER w)
             / (time_h - LAG(time_h) OVER w), 3) AS mu_per_h
FROM sample
WINDOW w AS (PARTITION BY run_id ORDER BY time_h)
ORDER BY run_id, time_h;
```

For run 1 this gives μ = 0.626, 0.616, 0.576 and 0.237 1/h over successive 2-hour intervals: exponential growth, then slowing as glucose runs out.

Save a query as a view so others can reuse it like a table:

```
CREATE VIEW run_summary AS
SELECT r.run_id, s.name AS strain, r.mode, r.reactor,
       MAX(sa.time_h) AS duration_h,
       MAX(sa.biomass_g_l) AS max_biomass_g_l,
       MAX(sa.product_g_l) AS final_titer_g_l
FROM run r
JOIN strain s       ON s.strain_id = r.strain_id
LEFT JOIN sample sa ON sa.run_id = r.run_id
GROUP BY r.run_id, s.name, r.mode, r.reactor;

SELECT * FROM run_summary;
```

Export results to CSV for Excel or plotting:

```
-- PostgreSQL
\copy (SELECT * FROM run_summary ORDER BY run_id) TO 'run_summary.csv' WITH (FORMAT csv, HEADER true)
```

```
# MySQL (tab-separated output from the client)
mysql -u lab_admin -p -e "SELECT * FROM run_summary" bioprocess > run_summary.tsv
```

7. Control access

Give the reader account read-only access.

PostgreSQL (run as `lab_admin` in the `bioprocess` database):

```
GRANT CONNECT ON DATABASE bioprocess TO lab_reader;
GRANT USAGE ON SCHEMA public TO lab_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lab_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO lab_reader;
```

MySQL (run as root):

```
GRANT SELECT ON bioprocess.* TO 'lab_reader'@'localhost';
FLUSH PRIVILEGES;
```

Test it: connected as `lab_reader`, `SELECT COUNT(*) FROM sample;` works (returned 10), while `DELETE FROM sample;` is refused with "permission denied for table sample" (PostgreSQL) or "DELETE command denied" (MySQL).

Security checklist:

- Use strong, unique passwords and never put them in shared scripts; use environment variables or a `.pgpass` / `.my.cnf` file with permissions 600.
- Allow connections only from the lab network (`pg_hba.conf` for PostgreSQL, the `'user'@'host'` part for MySQL).
- Turn on SSL/TLS for any connection over a network.
- For GMP or regulated data, add an audit trail (who changed what, when) and follow your 21 CFR Part 11 or ALCOA+ procedures.

8. Back up and restore

PostgreSQL:

```
# Back up (custom compressed format)
pg_dump -U lab_admin -Fc -f bioprocess_backup.dump bioprocess

# Restore into a new, empty database
createdb -U lab_admin bioprocess_restore
pg_restore -U lab_admin -d bioprocess_restore bioprocess_backup.dump
```

Tested: the restored copy held all 12 samples.

MySQL:

```
# Back up
mysqldump -u lab_admin -p --single-transaction --routines bioprocess > bioprocess_backup.sql

# Restore into a new, empty database
mysql -u root -p -e "CREATE DATABASE bioprocess_restore"
mysql -u root -p bioprocess_restore < bioprocess_backup.sql
```

Good practice: back up automatically every day (cron or Task Scheduler), keep copies off the server, and test a restore at least once a month. A backup you have never restored is not proven.

9. Connect from Python

Install the drivers: `pip install psycopg2-binary pymysql sqlalchemy pandas`.

Insert and read with a driver (PostgreSQL shown; for MySQL use `pymysql.connect(host="localhost", user="lab_admin", password="change_me", database="bioprocess")`):

```
import psycopg2

conn = psycopg2.connect(host="localhost", dbname="bioprocess",
                        user="lab_admin", password="change_me")
with conn, conn.cursor() as cur:            # commits automatically on success
    cur.execute(
        "INSERT INTO sample (run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l) "
        "VALUES (%s, %s, %s, %s, %s, %s)",
        (1, 12, 6.3, 2.52, 0.0, 1.7),
    )
    cur.execute("SELECT run_id, final_titer_g_l FROM run_summary ORDER BY run_id")
    print(cur.fetchall())
conn.close()
```

Always pass values as parameters (`%s`), never by building the SQL string yourself. This prevents SQL injection and handles types correctly.

Load a query straight into pandas for plotting or modeling:

```
import pandas as pd
from sqlalchemy import create_engine

# PostgreSQL
engine = create_engine("postgresql+psycopg2://lab_admin:change_me@localhost/bioprocess")
# MySQL
# engine = create_engine("mysql+pymysql://lab_admin:change_me@localhost/bioprocess")

df = pd.read_sql("SELECT * FROM sample WHERE run_id = 2 ORDER BY time_h", engine)
print(df[["time_h", "biomass_g_l", "product_g_l"]])
```

10. Troubleshooting

Problem	System	Likely cause	Fix
`connection refused`	Both	Server not running or wrong port	Start the service; check port 5432 / 3306
`password authentication failed` / `Access denied`	Both	Wrong password, or user exists only for another host	Reset the password; check `pg_hba.conf` or `'user'@'host'`
`permission denied for table`	PostgreSQL	Missing GRANT	Re-run the grants in Section 7
`violates check constraint`	Both	Impossible value in the data	Fix the value at the source; don't drop the constraint
`violates foreign key constraint`	Both	Sample points to a run that doesn't exist	Insert the run first
`duplicate key value`	Both	Same run and time point entered twice	Remove the duplicate row from the CSV
`LOAD DATA LOCAL INFILE` refused	MySQL	`local_infile` disabled	Start the client with `--local-infile=1` and set `local_infile=1` on the server
CHECK constraints ignored	MySQL	Version older than 8.0.16	Upgrade MySQL

Appendix A – Full PostgreSQL script

```
-- ============================================================
-- Bioprocess lab database: PostgreSQL schema, data and queries
-- Run:  psql -U lab_admin -d bioprocess -f bioprocess_postgres.sql
-- ============================================================

-- 1. Tables ---------------------------------------------------
CREATE TABLE strain (
    strain_id     SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL UNIQUE,
    organism      VARCHAR(100) NOT NULL,
    genotype      TEXT
);

CREATE TABLE run (
    run_id        SERIAL PRIMARY KEY,
    strain_id     INTEGER NOT NULL REFERENCES strain(strain_id),
    reactor       VARCHAR(50) NOT NULL,
    mode          VARCHAR(20) NOT NULL
                  CHECK (mode IN ('batch', 'fed-batch', 'continuous')),
    temperature_c NUMERIC(4,1) CHECK (temperature_c BETWEEN 0 AND 100),
    ph_setpoint   NUMERIC(3,1) CHECK (ph_setpoint BETWEEN 0 AND 14),
    started_at    TIMESTAMP NOT NULL,
    operator      VARCHAR(50)
);

CREATE TABLE sample (
    sample_id     SERIAL PRIMARY KEY,
    run_id        INTEGER NOT NULL REFERENCES run(run_id) ON DELETE CASCADE,
    time_h        NUMERIC(6,2) NOT NULL CHECK (time_h >= 0),
    od600         NUMERIC(6,3) CHECK (od600 >= 0),
    biomass_g_l   NUMERIC(6,3) CHECK (biomass_g_l >= 0),
    glucose_g_l   NUMERIC(6,3) CHECK (glucose_g_l >= 0),
    product_g_l   NUMERIC(6,3) CHECK (product_g_l >= 0),
    UNIQUE (run_id, time_h)
);

CREATE INDEX idx_sample_run_time ON sample (run_id, time_h);

-- 2. Data -----------------------------------------------------
INSERT INTO strain (name, organism, genotype) VALUES
  ('EC-WT',   'Escherichia coli', 'K-12 MG1655 wild type'),
  ('EC-SUC1', 'Escherichia coli', 'MG1655 delta-ldhA delta-pflB');

INSERT INTO run (strain_id, reactor, mode, temperature_c, ph_setpoint, started_at, operator) VALUES
```

```
  (1, 'BR-01', 'batch',     37.0, 7.0, '2026-10-01 09:00', 'A. Oyeodibo'),
  (2, 'BR-02', 'fed-batch', 37.0, 6.8, '2026-10-01 09:30', 'A. Oyeodibo');

INSERT INTO sample (run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l) VALUES
  (1, 0, 0.10, 0.04, 20.0, 0.0),
  (1, 2, 0.35, 0.14, 19.1, 0.1),
  (1, 4, 1.20, 0.48, 16.0, 0.4),
  (1, 6, 3.80, 1.52,  9.8, 1.0),
  (1, 8, 6.10, 2.44,  2.1, 1.6),
  (2, 0, 0.10, 0.04, 20.0, 0.0),
  (2, 4, 1.05, 0.42, 16.5, 0.9),
  (2, 8, 4.90, 1.96,  6.0, 3.8),
  (2, 12, 8.20, 3.28, 4.5, 7.2),
  (2, 16, 10.4, 4.16, 3.9, 10.5);

-- 3. Queries ------------------------------------------------
-- 3a. All samples with strain and run details (JOIN)
SELECT r.run_id, s.name AS strain, r.mode, sa.time_h, sa.biomass_g_l, sa.product_g_l
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
ORDER BY r.run_id, sa.time_h;

-- 3b. Final titer, glucose used and product yield per run (aggregates)
SELECT r.run_id, s.name AS strain,
       MAX(sa.product_g_l)                                      AS final_titer_g_l,
       MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l)                AS glucose_used_g_l,
       ROUND(MAX(sa.product_g_l) /
             NULLIF(MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l), 0), 3) AS yield_g_per_g
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
GROUP BY r.run_id, s.name
ORDER BY r.run_id;

-- 3c. Specific growth rate between samples (window function)
SELECT run_id, time_h, biomass_g_l,
       ROUND(LN(biomass_g_l / LAG(biomass_g_l) OVER w)
             / (time_h - LAG(time_h) OVER w), 3) AS mu_per_h
FROM sample
WINDOW w AS (PARTITION BY run_id ORDER BY time_h)
ORDER BY run_id, time_h;

-- 3d. A reusable view of run summaries
CREATE VIEW run_summary AS
SELECT r.run_id, s.name AS strain, r.mode, r.reactor,
       MAX(sa.time_h)      AS duration_h,
       MAX(sa.biomass_g_l) AS max_biomass_g_l,
       MAX(sa.product_g_l) AS final_titer_g_l
FROM run r
JOIN strain s      ON s.strain_id = r.strain_id
LEFT JOIN sample sa ON sa.run_id = r.run_id
GROUP BY r.run_id, s.name, r.mode, r.reactor;

SELECT * FROM run_summary ORDER BY run_id;
```

Appendix B – Full MySQL script

```
-- ============================================================
-- Bioprocess lab database: MySQL 8.0+ schema, data and queries
-- Run:  mysql -u lab_admin -p bioprocess < bioprocess_mysql.sql
-- (CHECK constraints need MySQL 8.0.16+, window functions 8.0+)
-- ============================================================

-- 1. Tables -------------------------------------------------
CREATE TABLE strain (
    strain_id     INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL UNIQUE,
```

```sql
    organism      VARCHAR(100) NOT NULL,
    genotype      TEXT
) ENGINE=InnoDB;

CREATE TABLE run (
    run_id        INT AUTO_INCREMENT PRIMARY KEY,
    strain_id     INT NOT NULL,
    reactor       VARCHAR(50) NOT NULL,
    mode          VARCHAR(20) NOT NULL
                  CHECK (mode IN ('batch', 'fed-batch', 'continuous')),
    temperature_c DECIMAL(4,1) CHECK (temperature_c BETWEEN 0 AND 100),
    ph_setpoint   DECIMAL(3,1) CHECK (ph_setpoint BETWEEN 0 AND 14),
    started_at    DATETIME NOT NULL,
    operator      VARCHAR(50),
    FOREIGN KEY (strain_id) REFERENCES strain(strain_id)
) ENGINE=InnoDB;

CREATE TABLE sample (
    sample_id     INT AUTO_INCREMENT PRIMARY KEY,
    run_id        INT NOT NULL,
    time_h        DECIMAL(6,2) NOT NULL CHECK (time_h >= 0),
    od600         DECIMAL(6,3) CHECK (od600 >= 0),
    biomass_g_l   DECIMAL(6,3) CHECK (biomass_g_l >= 0),
    glucose_g_l   DECIMAL(6,3) CHECK (glucose_g_l >= 0),
    product_g_l   DECIMAL(6,3) CHECK (product_g_l >= 0),
    UNIQUE (run_id, time_h),
    FOREIGN KEY (run_id) REFERENCES run(run_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE INDEX idx_sample_run_time ON sample (run_id, time_h);

-- 2. Data ------------------------------------------------
INSERT INTO strain (name, organism, genotype) VALUES
  ('EC-WT',   'Escherichia coli', 'K-12 MG1655 wild type'),
  ('EC-SUC1', 'Escherichia coli', 'MG1655 delta-ldhA delta-pflB');

INSERT INTO run (strain_id, reactor, mode, temperature_c, ph_setpoint, started_at, operator) VALUES
  (1, 'BR-01', 'batch',     37.0, 7.0, '2026-10-01 09:00:00', 'A. Oyeodibo'),
  (2, 'BR-02', 'fed-batch', 37.0, 6.8, '2026-10-01 09:30:00', 'A. Oyeodibo');

INSERT INTO sample (run_id, time_h, od600, biomass_g_l, glucose_g_l, product_g_l) VALUES
  (1, 0, 0.10, 0.04, 20.0, 0.0),
  (1, 2, 0.35, 0.14, 19.1, 0.1),
  (1, 4, 1.20, 0.48, 16.0, 0.4),
  (1, 6, 3.80, 1.52,  9.8, 1.0),
  (1, 8, 6.10, 2.44,  2.1, 1.6),
  (2, 0, 0.10, 0.04, 20.0, 0.0),
  (2, 4, 1.05, 0.42, 16.5, 0.9),
  (2, 8, 4.90, 1.96,  6.0, 3.8),
  (2, 12, 8.20, 3.28, 4.5, 7.2),
  (2, 16, 10.4, 4.16, 3.9, 10.5);

-- 3. Queries ---------------------------------------------
-- 3a. All samples with strain and run details (JOIN)
SELECT r.run_id, s.name AS strain, r.mode, sa.time_h, sa.biomass_g_l, sa.product_g_l
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
ORDER BY r.run_id, sa.time_h;

-- 3b. Final titer, glucose used and product yield per run (aggregates)
SELECT r.run_id, s.name AS strain,
       MAX(sa.product_g_l)                                    AS final_titer_g_l,
       MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l)              AS glucose_used_g_l,
       ROUND(MAX(sa.product_g_l) /
             NULLIF(MAX(sa.glucose_g_l) - MIN(sa.glucose_g_l), 0), 3) AS yield_g_per_g
FROM sample sa
JOIN run r    ON r.run_id = sa.run_id
JOIN strain s ON s.strain_id = r.strain_id
GROUP BY r.run_id, s.name
```

```sql
ORDER BY r.run_id;

-- 3c. Specific growth rate between samples (window function)
SELECT run_id, time_h, biomass_g_l,
       ROUND(LN(biomass_g_l / LAG(biomass_g_l) OVER w)
             / (time_h - LAG(time_h) OVER w), 3) AS mu_per_h
FROM sample
WINDOW w AS (PARTITION BY run_id ORDER BY time_h)
ORDER BY run_id, time_h;

-- 3d. A reusable view of run summaries
CREATE VIEW run_summary AS
SELECT r.run_id, s.name AS strain, r.mode, r.reactor,
       MAX(sa.time_h)      AS duration_h,
       MAX(sa.biomass_g_l) AS max_biomass_g_l,
       MAX(sa.product_g_l) AS final_titer_g_l
FROM run r
JOIN strain s      ON s.strain_id = r.strain_id
LEFT JOIN sample sa ON sa.run_id = r.run_id
GROUP BY r.run_id, s.name, r.mode, r.reactor;

SELECT * FROM run_summary ORDER BY run_id;
```